04. Station 3: Buffer & Log Shipper (`shipper.py`)

1. Overview & Purpose

`shipper.py` is the **network transport bridge (Shipper)**.

A common flaw in packet capture pipelines is executing HTTP requests directly inside the packet sniffer callback:

- Generating an HTTP POST to a remote API takes between **50ms and 300ms**.
- A standard 1 Gbps network interface processes thousands of packets per millisecond.
- If the sniffer waits for an HTTP response, it blocks. The OS kernel's raw socket buffer overflows, and live network packets are permanently dropped.

`shipper.py` solves this using a **Producer-Consumer pattern**:

1. **Producer (`enqueue_log`):** Runs on the packet capture thread. Pushing to an in-memory queue takes less than **1 microsecond**.
2. **Consumer (`shipper_worker`):** Runs in a separate background daemon thread. It pulls logs from the queue and sends HTTP POST requests without ever stalling packet capture.
3. **Fail-Safe Overflow:** If the SIEM server goes down or lags, the buffer caps at 1,000 items and drops incoming logs gracefully instead of exhausting system RAM.

2. Complete Source Code

```
python
from queue import Queue, Full
import threading
import time
import requests
import os

INGEST_URL = os.getenv("SIEM_INGEST_URL", "http://localhost:8431/api/v1/logs/ingest")
log_queue = Queue(maxsize=1000)

def enqueue_log(event):
    try:
        log_queue.put_nowait(event)
    except Full:
        print("[WARN] Queue full (1000 items). Event dropped. Check SIEM API status or increase buffer
size.")

def shipper_worker():
    while True:
        event = log_queue.get()
        try:
            requests.post(INGEST_URL, json=event, timeout=3)
        except requests.exceptions.RequestException:
            print("[ERROR] Request failed. Check your SIEM API connection.")
        finally:
            log_queue.task_done()

def start_shipper():
    thread = threading.Thread(target=shipper_worker, daemon=True)
    thread.start()
```

3. Function-by-Function Breakdown

1. `enqueue_log(event)`

- **Role:** Fast, non-blocking queue insertion (The Producer).
- **Execution:**
  - Called by `main.py` every time a packet yields a metadata dictionary.
  - Calls `log_queue.put_nowait(event)`. If the queue has space, the event is queued instantly.
  - If the queue has reached `maxsize=1000`, `put_nowait()` raises `Full`.
  - The `except Full:` block prints an actionable warning and discards that single log event so the capture loop continues uninterrupted.

2. `shipper_worker()`

- **Role:** Asynchronous background HTTP shipper (The Consumer).
- **Execution:**
  - Runs in a continuous `while True:` loop on a dedicated thread.
  - Calls `event = log_queue.get()`. Unlike `put()`, `get()` is designed to block when empty, using 0% CPU until a new packet arrives.
  - Executes `requests.post(INGEST_URL, json=event, timeout=3)`:
    - `json=event`: Serializes the metadata dictionary to standard JSON.
    - `timeout=3`: Guarantees that a hanging TCP connection to the backend will abort after 3 seconds instead of locking the worker indefinitely.
  - Catches `requests.exceptions.RequestException` to handle connection drops, DNS failures, or HTTP 500 errors without terminating the thread.
  - Calls `log_queue.task_done()` inside `finally:` for accurate queue accounting.

3. `start_shipper()`

- **Role:** Thread manager.
- **Execution:**
  - Instantiates `threading.Thread(target=shipper_worker, daemon=True)`.
  - `daemon=True` ensures the thread is marked as a background service; when the user presses `Ctrl+C` in `main.py`, Python terminates this thread instantly.
  - Starts the thread with `.start()`.

4. Line-by-Line Explanation

Line	Code	Explanation in Simple Terms
1	`from queue import Queue, Full`	Imports Python's built-in thread-safe FIFO queue and its buffer overflow exception.
2-5	`import threading`, `time`, `requests`, `os`	Standard libraries for concurrency, delays, HTTP requests, and environment variables.
7	`INGEST_URL = os.getenv(...)`	Reads `SIEM_INGEST_URL` from the OS environment, defaulting to `http://localhost:8431/api/v1/logs/ingest` for local dev.
8	`log_queue = Queue(maxsize=1000)`	Creates a FIFO buffer in RAM with a hard ceiling of 1,000 items to prevent memory leaks if the backend drops.
10	`def enqueue_log(event):`	Producer function called from the packet sniffer callback.
11-14	`try: log_queue.put_nowait(event) ...`	Attempts instantaneous insertion into RAM. Drops the item and logs a warning if the 1,000 item capacity is reached.
16	`def shipper_worker():`	Consumer function running in the background.
17-18	`while True: event = log_queue.get()`	Infinite loop pulling events from the queue. Blocks safely without spinning the CPU when the queue is empty.
19-20	`requests.post(INGEST_URL, json=event, timeout=3)`	Delivers the JSON log payload to the SIEM ingestion endpoint with a 3-second safeguard timeout.
21-22	`except requests.exceptions.RequestException:`	Catches network failures gracefully so the worker loop never crashes.
23-24	`finally: log_queue.task_done()`	Signals to the queue that the item was processed.
26-28	`def start_shipper(): ...`	Spawns and starts the daemon worker thread.

5. Variables & System Mapping

Variable	Type	Scope	Role in `shipper.py`	Downstream Mapping (Backend / Ingestion)
`INGEST_URL`	`str`	Global Module	The target SIEM HTTP endpoint.	Directly points to the backend route `POST /api/v1/logs/ingest`.
`log_queue`	`queue.Queue`	Global Module	Thread-safe FIFO buffer holding events awaiting HTTP transmission.	None. Purely internal in-memory buffer.
`event`	`dict`	Local	The single metadata dictionary passed from `ja4.py:process_packet()`.	Becomes the JSON body received by the backend ingestion API controller.
`thread`	`threading.Thread`	Local to `start_shipper`	Background thread handle.	None. Internal OS thread management.
`daemon=True`	`bool`	Argument	Marks the thread as a background worker.	None. Ensures clean terminal termination on `Ctrl+C`.